

小结

251

函数是命名了的计算单元，它对程序（哪怕是不大的程序）的结构化至关重要。每个函数都包含返回类型、名字、（可能为空的）形参列表以及函数体。函数体是一个块，当函数被调用的时候执行该块的内容。此时，传递给函数的实参类型必须与对应的形参类型相容。

在 C++ 语言中，函数可以被重载：同一个名字可用于定义多个函数，只要这些函数的形参数量或形参类型不同就行。根据调用时所使用的实参，编译器可以自动地选定被调用的函数。从一组重载函数中选取最佳函数的过程称为函数匹配。

术语表

二义性调用 (ambiguous call) 是一种编译时发生的错误，造成二义性调用的原因是在函数匹配时两个或多个函数提供的匹配一样好，编译器找不到唯一的最佳匹配。

实参 (argument) 函数调用时提供的值，用于初始化函数的形参。

Assert 是一个预处理宏，作用于一条表示条件的表达式。当未定义预处理变量 `NDEBUG` 时，`assert` 对条件求值。如果条件为假，输出一条错误信息并终止当前程序的执行。

自动对象 (automatic object) 仅存在于函数执行过程中的对象。当程序的控制流经过此类对象的定义语句时，创建该对象；当到达了定义所在的块的末尾时，销毁该对象。

最佳匹配 (best match) 从一组重载函数中为调用选出的一个函数。如果存在最佳匹配，则选出的函数与其他所有可行函数相比，至少在一个实参上是更优的匹配，同时在其他实参的匹配上不会更差。

传引用调用 (call by reference) 参见引用传递。

传值调用 (call by value) 参见值传递。

候选函数 (candidate function) 解析某次函数调用时考虑的一组函数。候选函数的名字应该与函数调用使用的名字一致，并且在调用点候选函数的声明在作用域之内。

constexpr 可以返回常量表达式的函数，一个 `constexpr` 函数被隐式地声明成内联函数。

默认实参 (default argument) 当调用缺少了某个实参时，为该实参指定的默认值。

可执行文件 (executable file) 是操作系统能够执行的文件，包含着与程序有关的代码。

函数 (function) 可调用的计算单元。

函数体 (function body) 是一个块，用于定义函数所执行的操作。

函数匹配 (function matching) 编译器解析重载函数调用的过程，在此过程中，实参与每个重载函数的形参列表逐一比较。

函数原型 (function prototype) 函数的声明，包含函数名字、返回类型和形参类型。要想调用某函数，在调用点之前必须声明该函数的原型。

隐藏名字 (hidden name) 某个作用域内声明的名字会隐藏掉外层作用域中声明的同名实体。

initializer_list 是一个标准类，表示的是一组花括号包围的类型相同的对象，对象之间以逗号隔开。

内联函数 (inline function) 请求编译器在可能的情况下在调用点展开函数。内联函数可以避免常见的函数调用开销。

链接 (link) 是一个编译过程，负责把若干

252

对象文件链接起来形成可执行程序。

局部静态对象 (local static object) 它的值在函数调用结束后仍然存在。在第一次使用局部静态对象前创建并初始化它，当程序结束时局部静态对象才被销毁。

局部变量 (local variable) 定义在块中的变量。

无匹配 (no match) 是一种编译时发生的错误，原因是在函数匹配过程中所有函数的形参都不能与调用提供的实参匹配。

对象代码 (object code) 编译器将我们的源代码转换成对象代码格式。

对象文件 (object file) 编译器根据给定的源文件生成的保存对象代码的文件。一个或多个对象文件经过链接生成可执行文件。

对象生命周期 (object lifetime) 每个对象都有相应的生命周期。块内定义的非静态对象的生命周期从它的定义开始，到定义所在的块末尾为止。程序启动后创建全局对象，程序控制流经过局部静态对象的定义时创建该局部静态对象；当 main 函数结束时销毁全局对象和局部静态对象。

重载确定 (overload resolution) 参见函数匹配。

重载函数 (overloaded function) 函数名与其他函数相同的函数。多个重载函数必须在形参数量或形参类型上有所区别。

形参 (parameter) 在函数的形参列表中声明的局部变量。用实参初始化形参。

引用传递 (pass by reference) 描述如何将实参传递给引用类型的形参。引用形参和其他形式的引用工作机制类似，形参被绑定到相应的实参上。

值传递 (pass by value) 描述如何将实参传递给非引用类型的形参。非引用类型的形参实际上是相应实参值的一个副本。

预处理宏 (preprocessor macro) 类似于内联函数的一种预处理功能。除了 assert 之外，现代 C++ 程序很少再使用预处理宏了。

递归循环 (recursion loop) 描述某个递归函数没有终止条件，因而不断调用自身直至耗尽程序栈空间的过程。

递归函数 (recursive function) 直接或间接调用自身的函数。

返回类型 (return type) 是函数声明的一部分，用于指定函数返回值的类型。

分离式编译 (separate compilation) 把一个程序分割成多个独立源文件的能力。

尾置返回类型 (trailing return type) 在参数列表后面指定的返回类型。

可行函数 (viable function) 是候选函数的子集。可行函数能匹配本次调用，它的形参数量与调用提供的实参数量相等，并且每个实参类型都能转换成相应的形参类型。

()运算符 (operator) 调用运算符，用于执行某函数。括号前面是函数名或函数指针，括号内是以逗号隔开的实参列表(可能为空)。

第 7 章

类

内容

7.1 定义抽象数据类型	228
7.2 访问控制与封装	240
7.3 类的其他特性	243
7.4 类的作用域	253
7.5 构造函数再探	257
7.6 类的静态成员	268
小结	273
术语表	273

在 C++ 语言中，我们使用类定义自己的数据类型。通过定义新的类型来反映待解决问题中的各种概念，可以使我们的编写、调试和修改程序。

本章是第 2 章关于类的话题的延续，主要关注数据抽象的重要性。数据抽象能帮助我们对象的实现与对象所能执行的操作分离开来。第 13 章将讨论如何控制对象拷贝、移动、赋值和销毁等行为，在第 14 章中我们将学习如何自定义运算符。